System Architecture Document

University Internship & Industry Matching Portal

---

1. Introduction

1.1 Purpose

This document outlines the system architecture of the University Internship & Industry Matching Portal, including its components, data flow, technology stack, and deployment structure. It is intended to guide developers, testers, and system administrators.

1.2 Scope

The system connects students, companies, and university administrators to manage internships, projects, startups, and competitions. It is a web-based platform deployed on Microsoft Azure with role-based portals.

---

2. Architecture Overview

The system follows a **3-tier architecture**:

1. **Presentation Layer (Frontend)**
2. **Application Layer (Backend/API)**
3. **Data Layer (Database and Storage)**

2.1 Presentation Layer (Frontend)

- **Technology:** React.js
- **Components:**
  - Student Portal
  - Company Portal
  - Admin Portal

- **Responsibilities:**
  - Display UI elements and dashboards
  - Handle user interactions
  - Communicate with backend via REST APIs

2.2 Application Layer (Backend/API)

- **Technology:** ASP.NET Core (.NET Web API)
- **Responsibilities:**
  - Handle business logic and workflows
  - Process student applications, project participation, and competition registration
  - Authenticate and authorize users
  - Serve data to frontend via RESTful endpoints

2.3 Data Layer (Database and Storage)

- **Database:** MySQL hosted on Azure Database Service
- **Storage:** Azure Blob Storage for resumes, project files, and other documents
- **Responsibilities:**
  - Store user profiles, applications, projects, competitions, and related data
  - Ensure data consistency and integrity

---

3. System Components

Component	Description
Student Portal	Interface for students to browse internships, projects, and competitions, apply, and track status
Company Portal	Interface for companies to review applicants, accept/reject candidates, and schedule interviews
Admin Portal	Interface for administrators to manage companies, internships, projects, competitions, and system monitoring

Component	Description
REST API	Backend APIs connecting frontend with the database and handling business logic
Database	Stores all data including users, applications, internships, projects, competitions
File Storage	Stores resumes, certificates, and project documents securely

---

4. Data Flow

1. **Student Interaction**
   - Students browse internships/projects/competitions
   - Apply via frontend
   - Data sent to backend APIs
   - Backend stores application in database and sends notifications
2. **Company Interaction**
   - Companies review applications via backend API
   - Accept/reject or schedule interviews
   - Status updated in database
   - Students notified via frontend
3. **Admin Interaction**
   - Admin adds/approves internships, projects, competitions
   - System validates and publishes data
   - Admin monitors system usage and generates reports

---

5. Deployment Architecture

- **Cloud Platform:** Microsoft Azure
- **Frontend Hosting:** Azure App Service or Static Web App
- **Backend Hosting:** Azure App Service (Web API)

- **Database Hosting:** Azure Database for MySQL
- **Storage:** Azure Blob Storage for documents
- **Security:** HTTPS, SSL, and role-based authentication

5.1 Deployment Diagram

```
[Student/Company/Admin Browsers]
           |
           v
   [React Frontend - Azure Hosting]
           |
           v
   [ASP.NET Core Web API - Azure App Service]
           |
           v
   [MySQL Database - Azure]   [Azure Blob Storage]
```

---

6. Security Architecture

- Role-based authentication (Student, Company, Admin)
- HTTPS communication for all frontend-backend interactions
- Secure storage for resumes and sensitive data
- Data validation and input sanitization

---

7. Scalability Considerations

- API designed to support multiple concurrent users
- Database normalization to improve performance
- Azure services provide horizontal scaling options
- Frontend optimized for responsiveness

---

8. Technologies Used

- Frontend: React.js
- Backend: ASP.NET Core Web API
- Database: MySQL
- Cloud Deployment: Microsoft Azure
- Storage: Azure Blob Storage
- Security: JWT / Role-based access control

---

9. Summary

The proposed architecture ensures separation of concerns, scalability, secure access, and maintainability. The 3-tier model enables independent development and deployment of frontend, backend, and data storage while integrating seamlessly via APIs.